FACULTY
OF MATHEMATICS
AND PHYSICS
Charles University

ABSTRACT OF DOCTORAL THESIS

Zdeněk Kasner

Data-to-Text Generation with Neural Language Models

Institute of Formal and Applied Linguistics

Supervisor: Mgr. et Mgr. Ondřej Dušek, Ph.D.

Study Program: Computational Linguistics

Prague 2024

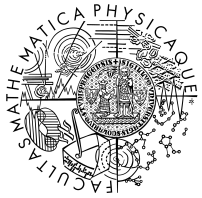
The results of this thesis were achieved in the period of a doctoral study at the Faculty of Mathematics and Physics, Charles University in years 2019–2024.

Doctoral Candidate:	Ing. Zdeněk Kasner Institute of Formal and Applied Linguistics
Supervisor:	Mgr. et Mgr. Ondřej Dušek, Ph.D. Institute of Formal and Applied Linguistics
Department:	Institute of Formal and Applied Linguistics, Faculty of Mathematics and Physics, Charles University Malostranské náměstí 25, 118 00 Prague 1, Czech Republic
Opponents:	prof. dr. Emiel Krahmer School of Humanities and Digital Sciences Department of Communication and Cognition Tilburg University Tilburg, Netherlands dr. Yaji Sripada School of Natural and Computing Sciences University of Aberdeen Aberdeen, United Kingdom
Chairman of Academic Council:	doc. RNDr. Pavel Pecina, Ph.D Institute of Formal and Applied Linguistics

This abstract was distributed on August 12, 2024.

The thesis defense will take place on September 5, 2024 at 9:30 a.m. in front of a committee for thesis defenses in the branch Mathematical Linguistics at the Faculty of Mathematics and Physics, Charles University, Malostranské nám. 25, Prague 1, room S1.

The thesis can be viewed at the Study Department of Doctoral Studies of the Faculty of Mathematics and Physics, Charles University, Ke Karlovu 3, Prague 2.



MATEMATICKO-FYZIKÁLNÍ
FAKULTA
Univerzita Karlova

AUTOREFERÁT DISERTAČNÍ PRÁCE

Zdeněk Kasner

Generování textu z dat s neuronovými jazykovými modely

Ústav formální a aplikované lingvistiky

Školitel: Mgr. et Mgr. Ondřej Dušek, Ph.D.

Studijní program: Matematická lingvistika

Praha 2024

Disertační práce byla vypracována na základě výsledků získaných během doktorského studia na Matematicko-fyzikální fakultě Univerzity Karlovy v letech 2019–2024.

Doktorand:	Ing. Zdeněk Kasner Ústav formální a aplikované lingvistiky
Školitel:	Mgr. et Mgr. Ondřej Dušek, Ph.D. Ústav formální a aplikované lingvistiky
Školicí pracoviště:	Ústav formální a aplikované lingvistiky, Matematicko-fyzikální fakulta, Univerzita Karlova Malostranské náměstí 25, 118 00 Praha 1, Česká republika
Oponenti:	prof. dr. Emiel Krahmer School of Humanities and Digital Sciences Department of Communication and Cognition Tilburg University Tilburg, Nizozemsko
	dr. Yaji Sripada School of Natural and Computing Sciences University of Aberdeen Aberdeen, Velká Británie
Předseda RDSO:	doc. RNDr. Pavel Pecina, Ph.D Ústav formální a aplikované lingvistiky

Autoreferát byl rozeslán dne 12. srpna 2024.

Obhajoba disertační práce se koná dne 5. září 2024 v 09:30 před komisí pro obhajoby disertačních prací v oboru Matematická lingvistika na Matematicko-fyzikální fakultě UK, Malostranské nám. 25, Praha 1, v místnosti S1.

S disertační prací je možno se seznámit na studijním oddělení Matematicko-fyzikální fakulty UK, Ke Karlovu 3, Praha 2.

1 Introduction

This thesis explores the intersection of data-to-text (D2T) generation and neural language models (LMs), focusing on how neural LMs can be leveraged to improve the quality and controllability of generated texts. Despite the successes of LMs in various areas, generating fluent and accurate text remains a challenging task, particularly when dealing with structured data.

Recent breakthroughs in neural LMs have led to significant improvements in text generation tasks, but these models often struggle with preserving the semantic accuracy of the output with respect to the input data. To address this challenge, we propose a novel approaches that use handcrafted or automatically extracted templates to transform structured data into a suitable input format for the models. We also propose how to restrict the role of the models within the D2T generation systems for better controllability. We demonstrate the effectiveness of our approach through a series of experiments on low-resource D2T generation tasks.

In addition to improving the quality of generated texts, we also investigate the evaluation of semantic accuracy in D2T generation. We develop novel automatic metrics that correlate strongly with human judgment, providing better ways to automate the evaluation of D2T generation systems. Furthermore, we examine the generalization performance of pretrained and large LMs on unseen domains and datasets, shedding light on the potential applications and limitations of these models.

Our research contributes to the advancement of D2T generation by developing novel methods for processing and visualizing structured data, evaluating the semantic accuracy of generated texts, and examining the behavior of neural language models in this context. By bridging the gap between traditional rule-based approaches and modern neural-based methods, our work aims to provide a more comprehensive understanding of the D2T generation process and its applications in various domains.

This thesis abstract contains (mostly extractive) summaries of the five thesis chapters describing our contributions. We also provide (mostly abstractive) summaries of the introduction and conclusion chapter. We refer the reader to the original thesis for detailed experimental background, results, and discussions.[1]

[1]The content of this section was generated by the `meta-llama/Meta-Llama-3-8B` model prompted with: "Here is an introduction chapter from a PhD thesis: `<content>`. Turn it into a shortened version for an extended abstract of a PhD thesis. Make it approximately 5 paragraphs.", using the original introduction chapter from the thesis as `<content>`. The thesis author manually post-edited the generated content and checked its factual accuracy.

2 Low-Resource Data-to-Text Generation

This chapter introduces three low-resource data-to-text (D2T) generation approaches based on pretrained language models (PLMs). The data we focus on are *RDF triples* from factual knowledge graphs in the WebNLG dataset (Gardent et al., 2017) and *key-value meaning representations* in the E2E dataset (Dušek et al., 2020, 2019).

2.1 Finetuning Pretrained Language Models

We first introduce a simple approach for generating knowledge graph descriptions. Our approach is based on finetuning a multi-lingual PLM on linearized graphs and the accompanying human-written descriptions from the WebNLG dataset. In the WebNLG+ 2020 Shared Task (Ferreira et al., 2020), our model ranked in the first third of the leaderboard for English and the first or second for Russian on automatic metrics.

Our input is a set of Resource Description Framework (RDF) triples $x \in X$, where each triple $x = (s, p, o)$ describes the relation p between the entities s and o in the knowledge graph. Our target output Y is a fluent and semantically accurate natural language description of X.

We formulate the task as *sequence-to-sequence* generation. First, we linearize the input sequence in the default order using two arbitrary separator tokens: one to delimit the triple constituents and another to delimit individual triples. Using the linearized sequence as an input and the target text as an output, we finetune a pretrained encoder-decoder model for the cross-entropy objective. With the finetuned model, we generate the target texts using autoregressive decoding.

2.1.1 Implementation

For the input data, we load and linearize the triples in their default order. Using the human-written references, we finetune the pre-trained `mbart.CC25` model from the FAIRSEQ toolkit (Ott et al., 2019) using the default parameters. We train a separate version of mBART for each language: mBART_{en} on English inputs and English outputs, and mBART_{ru} on English inputs and Russian outputs.

input	`Piotr_Hallmann | weight | 70.308` ▶ `Piotr_Hallmann | birthDate | 1987-08-25`
out (en)	Born on August 25th 1987, Piotr Hallmann has a weight of 70.308.
in	`Ciudad_Ayala | populationMetro | 1777539`
out (en)	The population metro of Ciudad Ayala is 1777539.
in	`Bakewell_tart | ingredient | Frangipane`
out (ru)	Франжипан - один из ингредиентов тарта Бейквелл.
transcr.	Franzhipan - odin iz ingredientov tarta Bejkvell.
transl.	Frangipane is one of the ingredients of the Bakewell tart.

Table 2.1: Example outputs from the mBART model finetuned for RDF-to-text generation. (1) The model can work with unseen entities, dates, and numbers. (2) The label deviates too much from its meaning for the unseen property `populationMetro`, leading to incorrect output. (3) The model trained on Russian targets can use English data to form sentences in Russian, transcribing the entities to Cyrillic.

		BLEU		**METEOR**		**ChrF++**		**BERTScore**		**BLEURT**	
All	Ours	50.34	(10)	0.398	(8)	0.666	(8)	0.951	(8)	0.57	(8)
	Baseline	40.57	(14)	0.373	(15)	0.621	(15)	0.943	(14)	0.47	(12)
Seen Cat.	Ours	59.13	(10)	0.422	(10)	0.712	(9)	0.960	(9)	0.58	(14)
	Baseline	42.95	(31)	0.387	(27)	0.650	(28)	0.943	(31)	0.41	(31)
Unseen Cat.	Ours	42.24	(10)	0.375	(13)	0.617	(10)	0.943	(11)	0.52	(10)
	Baseline	37.56	(12)	0.357	(15)	0.584	(15)	0.940	(12)	0.44	(12)
Unseen Ent.	Ours	51.23	(4)	0.406	(8)	0.687	(7)	0.959	(8)	0.63	(8)
	Baseline	40.22	(17)	0.384	(15)	0.648	(15)	0.949	(13)	0.55	(12)

Table 2.2: Results of mBART_{en} (all data, seen categories, unseen categories, unseen entities), compared to the baseline from the organizers. The numbers in brackets show the rank of each model (out of 35 submissions) with respect to the given metric.

2.1.2 Results

We report on WebNLG automatic and human evaluation results, as well as our error analysis.

The results of our approach for English are shown in Table 2.2. Our approach beats the baseline model based on the FORGe generator (Mille et al., 2019) in all metrics and places in the first third of the submissions. While it loses performance on unseen categories, the drop is less dramatic than other competing approaches. For Russian, the results are shown in Table 2.3. Our system not only beats the baseline by a large margin (as did all competing submissions), but it ranks first in two metrics out of four (BLEU, BERTScore) and second in the remaining ones.

The challenge organizers also ran a human evaluation campaign, asking annotators to rate the texts for data coverage, relevance, correctness, text structure, and fluency. Our English system achieved rank 1 for relevance, correctness and text structure, and rank 2 for data coverage and fluency. For Russian, our mBART_{ru} system ranks second for correctness and first (out of two to three clusters) in all other categories.

	BLEU		METEOR		ChrF++		BERTScore	
Ours	52.93	(1)	0.672	(2)	0.677	(2)	0.909	(1)
Baseline	23.53	(12)	0.461	(12)	0.511	(12)	0.836	(12)

Table 2.3: Results of mBART_{ru}, compared to the baseline. The numbers in brackets show the rank of each model (out of 12 submissions) if ordered by the given metric.

2.2 Iterative Sentence Fusion

In this section, we present an approach for generating semantically accurate texts from structured data in low-resource settings. Our approach builds on a text-editing model trained on the task of *sentence fusion*. After transforming individual data items to text using trivial templates, we iteratively improve the resulting text by applying sentence fusion, filtering, and re-ranking. Our approach achieves high levels of semantic accuracy due to the limited scope of the sentence fusion model and the guaranteed presence of the entities. We also demonstrate that our task formulation allows zero-shot D2T generation by training a model on a general-domain dataset for sentence fusion.

2.2.1 Method

We first collect a set of templates for each unique predicate. We use two approaches: (a) handcrafting the template manually for each predicate in the training set and (b) automatically extracting the template from the lexicalizations of the examples in the training set.

We train a model for the task of *sentence fusion*, i.e., combining sentences into a coherent text (Barzilay and McKeown, 2005). For re-ranking the text, we use an additional component for computing text fluency, which we refer to as LMScorer. We use perplexity of the text under a a PLM, computing the score of the output text Y composed of tokens $(y_1, \ldots, y_n)$ as a geometric mean of the token conditional probability.

The input of the algorithm (Figure 2.1) is a set of T ordered triples. First, we lexicalize the triple x_0 to get the output text Y_0 by filling the available templates and using the template with the best score from LMScorer. In each of the following steps $i = (1, \ldots, T-1)$, we lexicalize the triple x_i and concatenate it with Y_{i-1}. To improve the fluency of the text, we use the sentence fusion model with beam search to produce k hypotheses. We filter and re-rank the hypotheses (see the next paragraph), getting Y_i for the next step. The output is the text Y_{T-1} from the final step.

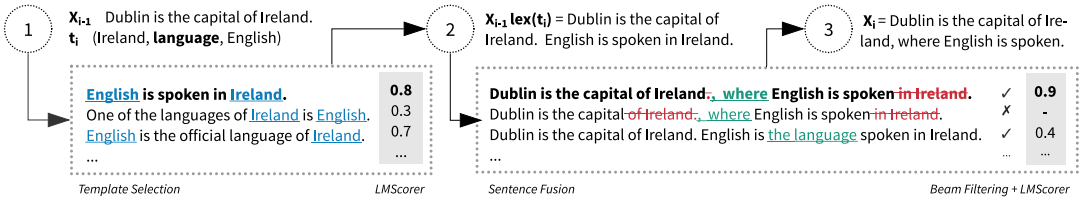


Figure 2.1: A single iteration of our algorithm for iterative D2T generation. In Step 1, the template for the triple is selected and filled. In Step 2, the sentence is fused with the template. In Step 3, the result for the next iteration is selected from the beam by filtering and language model scoring.

	WebNLG				E2E			
	BLEU	NIST	METEOR	ROUGE$_L$	BLEU	NIST	METEOR	ROUGE$_L$
baseline	0.277	6.328	0.379	0.524	0.207	3.679	0.334	0.401
sent. fusion	0.353	7.923	0.386	0.555	0.252	4.460	0.338	0.436
zero-shot	0.288	6.677	0.385	0.530	0.220	3.941	0.340	0.408
SFC	0.524	-	0.424	0.660	0.436	-	0.390	0.575
T5	0.571	-	0.440	-	-	-	-	-

Table 2.4: Results of automatic metrics on the WebNLG and E2E test sets.

2.2.2 Experiments

We experiment with the WebNLG and E2E datasets. We base our sentence fusion model on the text-editing model LaserTagger (Malmi et al., 2019), which is a PLM based on BERT (Devlin et al., 2019). As the LMScorer backend, we use the pre-trained GPT-2 language model (Radford et al., 2019) from the Huggingface Transformers (Wolf et al., 2019). We compute the perplexity scores using the *lm-scorer*[1] package.

For the *baseline*, we concatenate the best templates according to LMScorer without applying the sentence fusion (i.e., always using the fallback). For the *sentence fusion* experiments, we use LaserTagger with the autoregressive decoder with a beam of size 10. Additionally, we conduct a *zero-shot* experiment. We train the sentence fusion model with the same setup, but instead of the in-domain datasets, we use a subset of the *balanced-Wikipedia* portion of the DiscoFuse dataset (Geva et al., 2019).

2.2.3 Results

On lexical similarity metrics (Table 2.4), our system lags behind the state-of-the-art approaches selected for comparison: the Semantic Fidelity Classifier (SFC; Harkous et al., 2020) and the finetuned T5 model (T5; Kale and Rastogi, 2020). However, both the fusion and the zero-shot approaches show improvements over the baseline. It is

[1] https://github.com/simonepri/lm-scorer

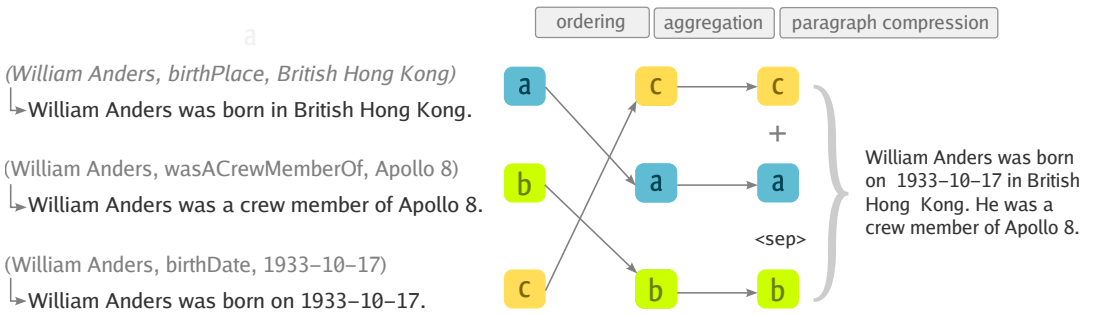


Figure 2.2: A scheme of our approach for zero-shot data-to-text generation from RDF triples. After a simple transformation of triples to facts, we apply the pipeline of modules for (1) ordering, (2) aggregation, and (3) paragraph compression. Individual modules are trained on a large general-domain text corpus and operate over text in natural language.

also important to note that our approach ensures zero entity errors by definition: we fill the entities verbatim into the templates, and if an entity is missing in the whole beam, we use a fallback instead (although semantic inconsistencies can still occur, e.g., if a verb or function words are missing).

The zero-shot model trained on DiscoFuse is able to correctly pronominalize or delete repeated entities and join the sentences with conjunctions, e.g. *William Anders was born in British Hong Kong, and was a member of the crew of Apollo 8.* While the model makes only limited use of sentence fusion, it makes the output more fluent while keeping strong guarantees of the output accuracy.

2.3 Pipeline of Text-to-Text Neural Modules

In this section, we propose using autoregressive PLMs and adding modules for ordering and aggregation, turning the generation process into a three-step pipeline (Sections 2.3.1 and 2.3.3). We also propose a way to make each of these steps trainable on a generic synthetic corpus (Section 2.3.2). We confirm that on WebNLG and E2E datasets, our approach can get lower rates of omissions and hallucinations than prior approaches according to a semantic accuracy metric while achieving levels of lexical similarity comparable to some of the prior systems; all of this without the need for in-domain training data (Sections 2.3.4 and 2.3.5).

2.3.1 Method

Similarly to Sections 2.1 and 2.2, we focus on the task of producing a natural language description Y a set of RDF triples $x \in X$, where each triple $x = (s, p, o)$ describes the relation p between the entities s and o in the knowledge graph.

	#train	#dev	#test	tok/src	tok/tgt	sent/src	sent/tgt
WebNLG	18,102	870	1,862	26.8	22.6	3.0	1.4
Clean E2E	33,236	4,299	1,847	29.2	22.3	4.2	1.5
WikiFluent-*full*	915,855	9,346	9,346	52.9	41.1	3.9	2.0
WikiFluent-*filtered*	700,517	7,149	7,149	45.6	35.4	3.4	1.8

Table 2.5: Number of examples (train / dev / test), the average number of tokens per source and target, the average number of sentences per source and target (after filling the templates for the D2T datasets), the total number of templates.

The first step in our pipeline involves transforming each of the input triples $x \in X$ into a fact $f \in F$ using a transformation $T : X \to F$. We define a fact f as a single sentence in natural language describing x.

The second step is the sentence ordering module $O(F)$, which produces an ordered sequence of facts: $F_o = \{f_{o_1}, \ldots, f_{o_n}\}$, where $o_{1:n}$ is a permutation of fact indices. An example outcome of such operation may be ordering adjacently facts mentioning *birth date* and *birth place* of a person, followed by their *occupation*, as it is shown in Figure 2.2. The ordering module allows downstream modules to focus only on operations over neighboring facts.

The third step is the aggregation module, that takes a sequence of ordered facts F_o as input and produces a sequence of sentence delimiters $A(F_o) = \{\delta_{o_1}, \delta_{o_2}, \ldots, \delta_{o_{n-1}}\}$; $\delta_i \in \{0, 1\}$.

Lastly, the paragraph compression (PC) module takes as input the ordered sequence of facts with delimiters $F_a = \{f_{o_1}, \delta_{o_1}, f_{o_2}, \ldots, \delta_{o_{n-1}}, f_{o_n}\}$ and produces the final text Y.

2.3.2 WikiFluent Corpus

We propose a way to build a large-scale synthetic corpus (WikiFluent) for training our modules from English Wikipedia. We first extracted 934k first paragraphs of articles from a Wikipedia dump[2] using WikiExtractor (Attardi, 2015). We used the first paragraphs of Wikipedia entries with lengths between 30-430 characters, filtering out lists, disambiguations, and malformed paragraphs. To balance the lengths of inputs, we divided the paragraphs according to their length into four equally-sized bins (30-130 characters, etc.) and selected 250k examples from each bin.

[2] `enwiki-20210401-pages-articles-multistream`

We train BART-base (Lewis et al., 2020) for (Narayan et al., 2017) on the WikiSplit corpus, containing human-made sentence splits from Wikipedia edit history (Botha et al., 2018). We split each paragraph into sentences using NLTK (Bird et al., 2009) and apply the split-and-rephrase model to each sentence. We also apply a coreference resolution model (Lee et al., 2018) from the AllenNLP framework (Gardner et al., 2018) and we replace referring expressions with their antecedents (e.g., pronouns with noun phrases).

To ensure that the generated sentences convey the same semantics as the original paragraph, we use the RoBERTa model[3] (Liu et al., 2019) finetuned on the MultiNLI dataset (Williams et al., 2018) for filtering the dataset.

2.3.3 Implementation

We transform triples into facts using a single-triple template t_i for each predicate, analogically to our approach in Section 2.2.2, i.e., using the templates extracted from the training data.

For our **ordering model**, we use the *Simple Pointer* model from Calizzano et al. (2021). The model is based on a pretrained BART-base model (Lewis et al., 2020) extended with a pointer network from Wang and Wan (2019). We train the model using the synthesized simple sentences in the WikiFluent corpus, randomly shuffling the order of the sentences and training the model to restore their original order.

We base our **aggregation model** on RoBERTa-large (Liu et al., 2019) with a token classification head. We input the sequence of ordered facts F_o into the model, separating each pair of facts f_{o_i} with a separator token. The token classification layer classifies each separator token into two classes $\{0, 1\}$ corresponding to the delimiter δ_i. We create training examples for aggregation using the synthesized sentences in the WikiFluent corpus, in which we set $\delta_i = 0$ for the sentences $i, i + 1$ which are the result of splitting a single sentence and $\delta_i = 1$ otherwise.

For the **paragraph compression model**, we finetune BART-base (Lewis et al., 2020) on the WikiFluent corpus, concatenating the split sentences on the input and training the model to produce the original, human-written complex sentences on the output. We add delimiters between the sentences i and $i + 1$ where $\delta_i = 1$ using a special token `<sep>`, which we add to the model vocabulary.

2.3.4 Experiments

We train our pipeline modules on the WikiFluent corpus as described in Section 2.3.3. Next, we use these modules *without any further finetuning* for generating descriptions for RDF triples on the WebNLG and E2E datasets.

[3] `https://huggingface.co/roberta-large-mnli`

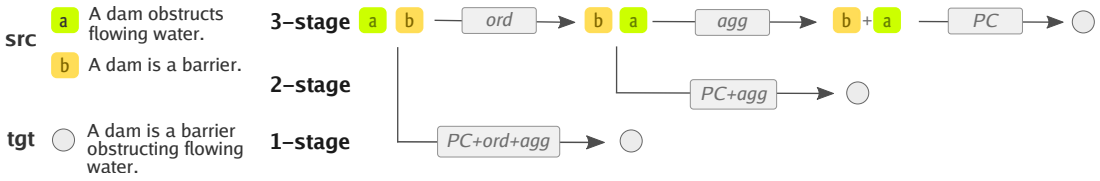


Figure 2.3: An example illustrating how the individual modules are trained and subsequently applied as the parts of the pipeline. See Section 2.3.4 for the description of the ordering model (ORD), the aggregation model (AGG), and the variants of the paragraph compression model (PC, PC+AGG, PC+ORD+AGG).

To evaluate individual components of our pipeline, we train three versions of the *paragraph compression* model (see Figure 2.3): PC, PC+AGG, and PC+ORD+AGG. The models share the same architecture and targets but differ in their inputs. Correspondingly, we test three versions of the pipeline for the ablation study: 3-STAGE, 2-STAGE, and 1-STAGE.

2.3.5 Evaluation

		WebNLG				E2E			
		B	**M**	**O**	**H**	**B**	**M**	**O**	**H**
COPY		37.18	38.77	0.000	0.000	24.19	34.89	0.000	0.000
UPF-FORGe*		38.65	39.00	0.075	0.101	-	-	-	-
MELBOURNE*		45.13	37.00	0.237	0.202	-	-	-	-
Ke et al. (2021)†*		66.14	47.25	-	-	-	-	-	-
Laha et al. (2019)†		24.80	34.90	-	-	-	-	-	-
TGEN*		-	-	-	-	40.73	37.76	0.016	0.083
Harkous et al. (2020)†*		-	-	-	-	43.60	39.00	-	-
full	3-STAGE	42.92	39.07	0.051	0.148	**36.04**	36.95	**0.001**	**0.001**
	2-STAGE	42.90	39.28	**0.043**	0.125	35.84	36.91	**0.001**	**0.001**
	1-STAGE	39.08	38.94	0.071	0.204	30.81	36.01	0.009	0.122
filtered	3-STAGE	43.19	39.13	0.152	**0.073**	35.88	36.95	**0.001**	**0.001**
	2-STAGE	**43.49**	**39.32**	0.146	0.096	36.01	**36.99**	**0.001**	**0.001**
	1-STAGE	42.99	38.81	0.202	0.093	34.08	36.32	0.012	0.050

Table 2.6: Automatic metrics on the WebNLG and E2E datasets. B = BLEU, M = METEOR, O = omissions / # facts, H = hallucinations / # examples. The systems marked with asterisk (*) are trained on in-domain data. The results for the systems marked with † are taken from the respective works. **Boldface** denotes the best variant of our zero-shot system.

		WebNLG					E2E				
		H	**I**	**O**	**R**	**G**	**H**	**I**	**O**	**R**	**G**
full	3-STAGE	3	39	2	2	16	0	1	0	0	17
	2-STAGE	8	36	1	5	16	1	1	0	1	23
	1-STAGE	28	27	6	10	20	17	0	1	79	45
filtered	3-STAGE	2	37	2	1	15	0	0	0	0	17
	2-STAGE	5	32	1	2	14	0	0	0	0	11
	1-STAGE	8	40	6	6	16	11	2	1	41	22

Table 2.7: Number of manually annotated errors on 100 examples: H = hallucinations, I = incorrect fact merging, O = omissions, R = redundancies, G = grammar errors or disfluencies.

Automatic Metrics The automatic evaluation (Table 2.6) shows that our systems consistently outperform the COPY baseline (e.g., ∼12 BLEU points for E2E), which is already strong thanks to our manually curated set of templates.[4] Automatic scores also suggest that our systems are comparable with some older supervised systems, although they underperform the state-of-the-art supervised systems.

The 2-STAGE system is generally on par with the 3-STAGE system, which indicates that explicit aggregation using the AGG model may not be necessary. However, a separate aggregation module allows one to control the aggregation step explicitly. The models using the filtered version of the corpus generally produce better results, although they also bring in a larger number of omissions.

Error Analysis We also manually examined 100 model outputs, counting the number of semantic errors (hallucinations, omissions, incorrect fact merging, redundancies) and grammatical errors. The 1-STAGE model (which has to order the facts implicitly) tends to repeat the facts in the text (especially in E2E) and produces frequent hallucinations. These problems are largely eliminated with the 2-STAGE and 3-STAGE models, which produce almost no hallucinations or omissions.

However, the outputs on WebNLG for all systems suffer from semantic errors resulting from merging unrelated facts. On the E2E data, where predicates share the same subject, the outputs are generally consistent, and the 2-STAGE and 3-STAGE models exhibit almost no semantic errors.

[4]On WebNLG, COPY achieves 37.18 BLEU points, compared to 24.80 BLEU points of the *full system* of Laha et al. (2019), which uses automatic template generation.

3 Evaluating Semantic Accuracy

This chapter introduces two metrics for evaluating the semantic accuracy of data-to-text (D2T) generation. The metrics are predominantly based on pretrained language models (PLMs) and generic text-to-text operations, which makes our approaches applicable to various tasks and domains.

3.1 Detecting Omissions and Hallucinations

In this section, we propose a new metric for evaluating the semantic accuracy of D2T generation. Our metric is based on a neural model pretrained for natural language inference (NLI).

3.1.1 Method

We are given a set of RDF triples $x \in X$, where each triple $x = (s, p, o)$ describes the relation p between the entities s and o, and the corresponding natural language description Y. Our task is to assess whether Y mentions all the triples in X. Additionally, we should also check whether the text mentions any extra information.

We used simple templates for transforming individual triples to facts, i.e., simple sentences capturing the triple meaning, similarly to Chapter 2.

Checking Semantic Accuracy with NLI We use an NLI model for assessing the semantic accuracy of generated texts. Consider the two input facts from Figure 3.1: $F =$ {*"Blue spice is a pub"*, *"Blue Spice is located in the riverside"*} and the generated text: $Y =$ *"You can bring your kids to Blue Spice in the riverside area."* We propose using an NLI model for checking if the semantic information implied by F and Y is equal. In this case, the model should detect an omission, i.e., that the first fact is not entailed by the text (there is no mention of Blue Spice being a pub), and also a hallucination, i.e., that the text is not entailed by the facts (the information about kids is superfluous).

We achieve this by using the NLI model to check for entailment in two directions:

(1) To check for omissions, we use the whole generated text as a premise and sequentially feed each fact as a hypothesis to the NLI model. Any failed NLI check is considered an omission.

Figure 3.1: An example of evaluating the output from a D2T system with our metric. The generated text is used as a *premise* (P) to check for omissions and as a *hypothesis* (H) to check for hallucinations. The NLI model generates probabilities for *contradiction* (C), *neutral* (N) and *entailment* (E).

(2) To check for hallucinations, we concatenate all facts as a premise and feed the generated text as a hypothesis to the NLI model. If this NLI check fails, the text is considered to contain hallucination.

The final output of our metric is either 4-way (*OK*, *omission*, *hallucination*, or *omission+hallucination*; denoted as FINE) or 2-way (*OK* or *not_OK*; denoted as ROUGH). Additionally, we compute a *confidence score* of the model as the minimum of all the entailment probabilities.

3.1.2 Experiments

For our NLI model, we use the `roberta-large-mnli`[1] checkpoint of the pretrained RoBERTa model (Liu et al., 2019). We use the model *as is*, without any further training on domain-specific data. Given a premise text and a hypothesis text, the NLI model produces a probability distribution over three results: *contradiction*, *neutral*, and *entailment* (see Section 3.1.1). We consider a NLI check as passed if the probability for *entailment* is the highest of the three.

We experiment with the WebNLG and E2E datasets. Since both datasets were used in shared tasks, we compare the outputs of our system with the respective measures of semantic accuracy.

We experiment with the *Default* and *Backoff* approaches to transforming triples to facts, as described in Section 3.1.1.

[1] `https://huggingface.co/roberta-large-mnli`

	WebNLG					E2E				
	A	**R**	**P**	**F1**	ρ	**Af**	**Ar**	**R**	**P**	**F1**
Default	0.775	0.772	0.796	0.784	0.628	0.911	0.933	0.895	0.910	0.903
Backoff	0.768	0.760	0.793	0.776	0.637	0.846	0.874	0.913	0.768	0.834

Table 3.1: WebNLG and E2E results, compared to crowdsourced human ratings and the automatic evaluation script, respectively (A = accuracy, Af = FINE accuracy, Ar = ROUGH accuracy, R = recall, P = precision, F1 = F-measure, ρ = Spearman correlation of confidence scores with human scores).

3.1.3 Evaluation

We evaluate our metric in terms of accuracy (**A**), precision (**P**), recall (**R**), and F1-measure (**F1**) with respect to the corresponding ground truth outputs. For WebNLG, we additionally compute Spearman correlation coefficient (ρ) of the model's confidence scores with the average human scores. For E2E, we evaluate the accuracy for both the FINE (**Af**) and ROUGH (**Ar**) variants described in Section 3.1.1, making use of the fact that the automatic script reports both omissions and hallucinations. The scores for both datasets are summarized in Table 3.1.

We additionally perform a manual error analysis on a random sample of 100 error examples for each dataset, i.e., examples where our metric gave a different assessment from the ground truth. Our re-examination shows that almost half of the error examples (42 out of 100) were in fact correctly classified by our metric (i.e., their crowdsourced human annotation was incorrect), so the true performance is most likely higher than the reported numbers. The rest of the results and the discussion are available in the thesis.

3.2 Token-Level Error Classification

In this section, we present an automatic metric for fine-grained detection of semantic accuracy errors in D2T generation outputs. The metric combines a rule-based D2T generation system and PLMs (Section 3.2.2).

3.2.1 Shared Task in Evaluating Accuracy

Our metric was a submission to the the Shared Task on Evaluating Accuracy in Generated Texts at INLG 2021. The goal of the task was to develop a token-level error annotation metric for complex D2T generation (Reiter and Thomson, 2020). The organizers of the shared task first manually annotated 60 outputs of various neural systems trained on Rotowire, using the error types defined in Thomson and Reiter (2020):

- **NUMBER^{N}** – Incorrect number (both digits and numerals).

Figure 3.2: Rule-based system that we use to generate facts from the input data. The facts are used as input to the error-checking model (see Figure 3.3). We experiment with (a) simple hand-crafted templates and (b) compact sentences generated by the FORGe system.

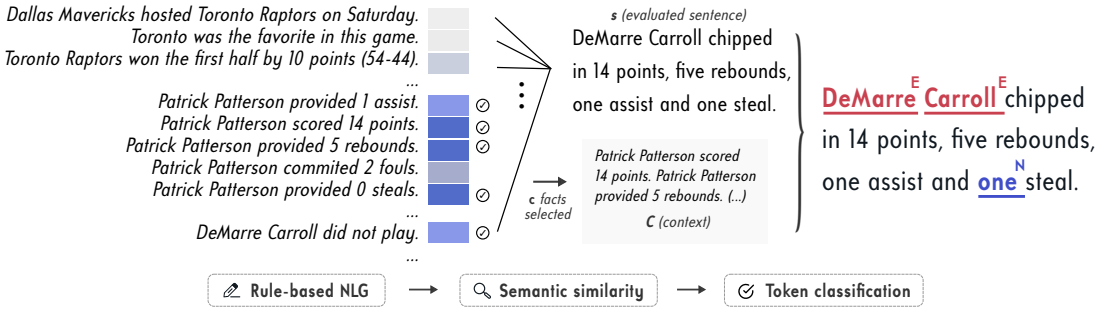


Figure 3.3: An overview of our system. First, we generate the facts from the input table with a rule-based NLG system (see Figure 3.3). For each evaluated sentence s, we select c facts with the highest semantic similarity, getting a context C. The pair (C, s) is given as an input to a pretrained LM for token-level error classification.

- NAME^{E} – Incorrect named entity (people, places, teams, days of the week).
- WORD^{W} – Incorrect word which is not one of the above.
- CONTEXT^{C} – A phrase inappropriate for the context.
- NOT_CHECKABLE^{NC} – A statement which cannot be checked.
- OTHER^{O} – Any other type of mistake.

3.2.2 Our System

We first use a rule-based system to generate facts G from the input tables in natural language. For each evaluated sentence s and for each fact $g_i \in G$, we embed the sentence tokens by applying mean pooling on the output of `paraphrase-distilroberta-base-v2`, getting the embedding vectors e_s and e_{g_i}. For the context C, we select the top c sentences from G that have the highest cosine similarity to s.

Next, we use an error tagger based on RoBERTa (Liu et al., 2019) with a token-level classification head. The model receives an input $X = (C, s)$ and is trained to annotate each token in s either with an *OK* label or with a label corresponding to one of the error categories. We experiment with two data sources for training the model:

Error Type	Mistake		Token	
	R	P	R	P
NAME^{E}	0.750	0.846	0.759	0.862
NUMBER^{N}	0.777	0.750	0.759	0.752
WORD^{W}	0.514	0.483	0.465	0.529
CONTEXT^{C}	0.000	-	0.000	-
NOT_CHECKABLE^{NC}	0.000	-	0.000	-
OTHER^{O}	0.000	-	0.000	-
Overall	0.691	0.756	0.550	0.769

Table 3.2: Results of our system on test data: recall (R) and precision (P) are shown for individual error types.

- *gold-standard annotated data* from the shared task (which contains all error types),
- *synthetic data* created by perturbing the human-written summaries from Rotowire (which contains only NAME^{E} and NUMBER^{N} errors).

3.2.3 Experiments

For the error tagger, we train a PyTorch version of RoBERTa from the Huggingface Transfomers repository (Wolf et al., 2019). We compute recall and precision of the model output with respect to the human-annotated data. Since we use the human-annotated data for training, we perform 6-fold cross-validation: in each run, we use 45 games for training, 5 games for validation, and 10 games for evaluation. For our final submission, we selected the model with the best F1-score overall, which is 0.65 (0.61 recall and 0.69 precision).

Table 3.2 shows the results of our model on the official test data of the task, broken down by error types. The overall scores are higher than on the development set – test set recall is 0.69 (vs. 0.61 on the development set), and precision is 0.76 (vs. 0.69). We note that our model was able to identify only three types of errors (NAME^{E}, NUMBER^{N}, WORD^{W}), having better results for the NAME^{E} and NUMBER^{N} errors.

4 Unified Data Processing

In this chapter, we introduce our TabGenie toolkit. We first convert the input data in sixteen data-to-text (D2T) generation datasets of various formats and provenance into a standard tabular format. On top of the unified format, we build a web interface, command-line interface and Python bindings for simplifying data visualization and processing.

4.1 TabGenie Toolkit

The most commonly used D2T generation datasets contain three high-level input data formats: tables, RDF triples, and key-value pairs. We note that converting the latter two to tabular format requires only minimal changes to the data structure while allowing a unified data representation and visualization. Therefore, input data in TabGenie is in tabular format.

To better accommodate the format of datasets such as ToTTo (Parikh et al., 2020) or HiTab (Cheng et al., 2022), we also allow individual cells to be *highlighted* – i.e., preselected for generating an output description.

4.1.1 Datasets

We include the 16 D2T generation dataset available under an open-source license. To ease the data distribution, we load all the datasets using the Huggingface `datasets` package (Lhoest et al., 2021). A custom dataset can be added to TabGenie by simply sub-classing the data loader class and overriding the method for processing individual entries.

4.1.2 Web Interface

TabGenie offers a way to interact with datasets through a *web interface*. The interface features a single-page layout with three panels containing user controls, input data, and system outputs (see Figure 4.1). TabGenie provides better visualizations than existing data viewers, especially in the case of large and hierarchical tables.

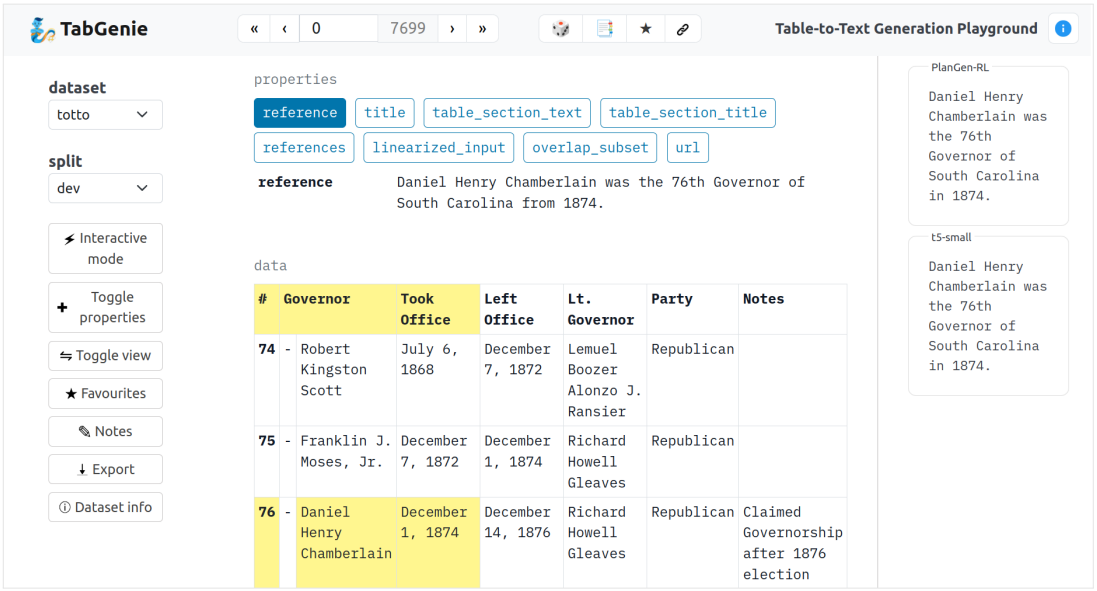


Figure 4.1: The web interface of TabGenie. The *left panel* and the *top navigation bar* contain user controls; the *center panel* shows table properties and table content; the *right panel* contains system outputs.

In the interactive mode, the user can modify the input data and observe how changes influence model outputs. We assume that the model provides access through a simple API queried by TabGenie. The user can highlight different cells, edit cell contents, and edit the parameters of the downstream processor.

TabGenie also allows to visualize static pre-generated outputs, which are loaded in a JSONL[1] format from a specified directory. Multiple outputs can be displayed alongside a specific example for comparing outputs from multiple systems.

Besides the web interface, TabGenie also provides developer-friendly access through Python bindings and a command-line interface. Both of these interfaces aim to simplify dataset preprocessing in downstream tasks. The key benefit of using TabGenie is that it provides streamlined access to data in a consistent format, removing the need for dataset-specific code for extracting information such as table properties, references, or individual cell values.

4.1.3 Implementation

TabGenie runs with Python >=3.8 and requires only a few basic packages as dependencies. It can be installed as a stand-alone `tabgenie` module from PyPI or from the project repository.

In the thesis, we also present several case studies for using TabGenie in D2T generation research. The instructions and code samples for these tasks are available in the project repository.

[1] `https://jsonlines.org`

5 Examining Model Behavior

In this chapter, we analyze generalization abilities of neural language models (LMs). Specifically, we investigate how well LMs used for data-to-text (D2T) generation generalize to the data with domains or formats that the models were not specifically trained for. To help us with the investigation, we introduce custom datasets that we collect for the purposes of our experiments.

5.1 Describing Relations in Knowledge Graphs

In this section, we investigate how human-readable data labels help pretrained language models (PLMs) with D2T generation. For our experiments, we first collect a large set of relations from three large-scale knowledge graphs (KGs) (Wikidata, DBpedia, and YAGO). For each relation, we collect its label, textual description, and up to five triples in which the relation occurs in the KG. We then use human annotators to collect a *verbalization* for each triple, i.e., a short sentence capturing the meaning of the triple. After filtering, our dataset—which we call Rel2Text (Re-writing edge labels to Text)[1]—contains 4,097 single triples covering 1,522 unique relations.

5.1.1 Analysis and Experiments

Using the Rel2Text dataset, we investigate the following questions:

- **(a)** Are PLMs finetuned for D2T generation able to describe relations *not present in the finetuning corpus*?
- **(b)** How many *training examples* do the PLMs need to generate satisfactory outputs?
- **(c)** How do the PLMs behave when provided *limited lexical cues* about the relation?
- **(d)** Can relation *descriptions* help to clarify ambiguous cases and improve the semantic accuracy of the outputs?

[1] Or simply "Relations-to-Text".

To answer the questions, we use the REL2TEXT *test set* to evaluate a finetuned BART model (Lewis et al., 2020). We train the models on the REL2TEXT, WebNLG (Ferreira et al., 2020; Gardent et al., 2017), and KeLM (Agarwal et al., 2021) datasets, all of which focus on verbalizing factual information from KGs and use the same triple-based input data format.

In our experiments, we compare the following systems:

- **Copy Baseline**: We introduce a simple *copy* baseline by outputting the triple constituents separated by space: "`[subject] [relation] [object]`".
- **Full Training Data**: We finetune the models on the aforementioned datasets, denoting the trained models *full-rel2text*, *full-webnlg*, and *full-kelm*, respectively.
- **Limited Training Data**: We finetune the *fewshot-N* (where $N = \{25, 50, 100, 200\}$) models for 10 epochs without validation.
- **Limited Lexical Cues**: We mask the relation labels, considering the scenarios *mask-test* (masking relations only during test time), *mask-train* (masking relations only during train time), and *mask-all* (both cases at once).
- **Incorporating Descriptions**: We consider two scenarios: *desc-repl* (replacing a label with its description) and *desc-cat* (concatenating the label and the description.

5.1.2 Evaluation Setup

We evaluate generated outputs using automatic metrics from the GEM-metrics[2] package (Gehrmann et al., 2021). We also identify four sources of model errors, along with two types of input data errors (see examples in Table 5.1) and manually identify them in the sample of 100 examples.

Table 5.2 shows automatic scores for all our models, examples of model outputs for each error type are shown in Table 5.1. We provide a detailed discussion of the results in the thesis. We also show how a model trained on REL2TEXT can be applied to two downstream tasks.

[2] `https://github.com/GEM-benchmark/GEM-metrics`

	Label	Example inputs and outputs (✗ incorrect, ✓ correct)
model	Sem	*(Yousra Matine, sport country, Morocco)* ✗ Yousra Matine was born in Morocco. [*mask-mask*] ✓ Yousra Matine plays for Morocco. [*full-rel2text*]
	Dir	*(Kentucky Channel, former broadcast network, KET ED)* ✗ KET ED was broadcast on Kentucky Channel ED. [*fewshot-100*] ✓ The Kentucky Channel was broadcast on KET ED. [*full-rel2text*]
	Lit	*(Vietnam Television, first air date, 1970-09-07)* ✗ The first air date of Vietnam Television was 1970-09-07. [*full-kelm*] ✓ Vietnam Television first aired on 1970-09-07. [*full-rel2text*]
	Lex	*(RPG-43, used in war , The Troubles)* ✗ RPG-43 was used in the The Troubles. [*full-rel2text*] ✓ The RPG-43 was used in the Troubles. [*full-kelm*]
data	Ent	*(The Age of Entitlement, by artist, The Basics)* ✗ The Age of Entitlement was written by The Basics. [*full-kelm*] ✓ The Age of Entitlement was recorded by The Basics. [*full-rel2text*]
	Lbl	*(General Motors Epsilon platform, vehicle, Cadillac XTS)* ✗ General Motors Epsilon is a vehicle similar to the Cadillac XTS. [*full-webnlg*] ✓ General Motors Epsilon platform is used in the Cadillac XTS. [*desc-cat*]

Table 5.1: Error categories used in manual analysis, with examples of errors found and corresponding correct verbalizations (square brackets denote the model). Model error types (top): Sem – The output is semantically incorrect, Dir – The direction of the relation is swapped, Lit – The verbalization is too literal, Lex – There is a lexical error in the output. Input data error types (bottom): Ent – The verbalization may depend on the entities, Lbl – The relation label is not clear.

	Lexical			Semantics					Referenceless				
	BLEU	**MET**	**BLR**	**SS**	**C**	**N**	**E**	**NB**	**U-1**	**CE-2**	**TTR**	**PPL**	**len**
human	-	-	-	-	-	-	-	-	1785	2.13	0.62	5.88	9.55
copy	29.04	37.52	0.09	4.79	1.22	7.57	91.21	0.74	1606	1.17	0.7	7.55	6.72
full-rel2text	52.54	44.86	0.54	4.72	3.50	4.65	91.85	0.88	1661	1.96	0.58	5.89	9.16
full-webnlg	41.99	41.59	0.41	4.65	3.68	6.93	89.39	0.86	1651	2.54	0.56	5.65	10.29
full-kelm	46.74	42.94	0.46	4.70	3.95	5.29	90.77	0.86	1652	2.32	0.56	5.83	9.71
fewshot-25	31.13	35.52	-0.02	3.94	8.35	27.26	64.39	0.65	1445	2.93	0.52	5.34	10.67
fewshot-50	40.60	40.05	0.25	4.44	8.04	13.12	78.84	0.76	1536	2.31	0.55	5.79	9.90
fewshot-100	45.88	42.38	0.38	4.53	6.34	10.60	83.06	0.81	1600	2.13	0.57	5.85	9.57
fewshot-200	48.67	43.34	0.44	4.58	5.40	9.03	85.57	0.83	1626	2.04	0.58	5.89	9.36
mask-test	42.45	38.52	0.25	3.99	14.91	18.47	66.62	0.65	1669	1.96	0.61	5.69	8.96
mask-train	46.90	43.15	0.43	4.55	5.85	11.55	82.61	0.81	1646	2.00	0.57	5.91	9.74
mask-all	42.53	38.49	0.24	3.85	17.58	25.15	57.26	0.61	1677	1.96	0.61	5.66	9.16
desc-repl	49.35	42.85	0.47	4.57	5.78	8.80	85.42	0.82	1693	1.94	0.59	5.86	9.18
desc-cat	53.07	45.04	0.55	4.72	3.46	4.66	91.88	0.87	1668	1.91	0.59	5.92	9.11

Table 5.2: The summary of evaluation using automatic metrics on Rel2text test set. **MET** = METEOR, **BLR** = BLEURT, **TTR** = MSTTR. See Section 5.1.1 for the descriptions of the models and metrics.

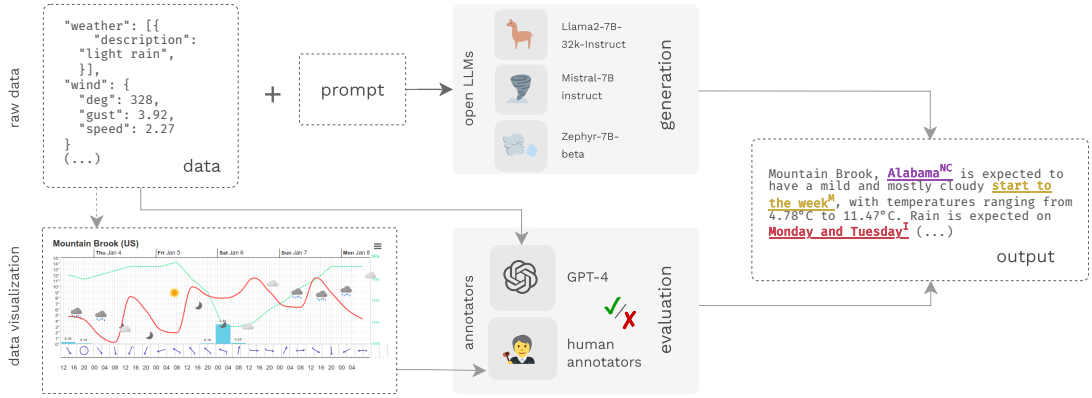


Figure 5.1: Our experimental setup. We first generate the outputs using LLMs that are given raw data and a task-specific prompt. We annotate the token-level semantic errors in the LLM outputs with (a) an automatic metric based on GPT-4 that matches the output to the raw data, and (b) human annotators, who annotate the errors in the output given the data visualization.

5.2 Data-to-Text Generation with Large Language Models

In this section, we look into D2T generation with large language models (LLMs).

5.2.1 Reference-Free D2T Generation

We introduce a tool named QUINTD for collecting ad-hoc test sets from public APIs for five D2T generation tasks: generating five-day weather forecasts, product descriptions, ice hockey game summaries, chart captions, and entity descriptions. The tasks are based on structured data in common formats: JSON, CSV, and Markdown.

Using QUINTD, we collected QUINTD-1 – a dataset with 100 examples per domain for each split. Each example in QUINTD-1 consists of a structured data record x. Given x and a prompt $\mathcal{P}$, the goal of LLMs is to generate natural language output y faithful to the data x, according to the instructions in the prompt $\mathcal{P}$.

5.2.2 Experiments

Models For our experiments, we selected the following LLMs available under an open license: Llama 2 (Touvron et al., 2023; TogetherAI, 2023),[3] Mistral (Jiang et al., 2023),[4] and Zephyr (Tunstall et al., 2023).[5] For the final experiments, we also included GPT-3.5 (`gpt-3.5-turbo-1106`) accessed through the OpenAI API.

[3] `https://huggingface.co/togethercomputer/Llama-2-7B-32K-Instruct`
[4] `https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.1`
[5] `https://huggingface.co/HuggingFaceH4/zephyr-7b-beta`

	Incorrect		Not Checkable		Misleading		Other		All cat.		
	$\mathcal{E}_{\text{hum}}$	$\mathcal{E}_{\text{gpt}}$	$\mathcal{E}_{\text{hum}}$	$\mathcal{E}_{\text{gpt}}$	$\mathcal{E}_{\text{hum}}$	$\mathcal{E}_{\text{gpt}}$	$\mathcal{E}_{\text{hum}}$	$\mathcal{E}_{\text{gpt}}$	$\mathcal{E}_{\text{hum}}$	$\mathcal{E}_{\text{gpt}}$	**Tok.**
Llama 2	1.57	**2.79**	1.25	0.91	0.25	**0.12**	**0.10**	0.09	3.18	3.90	83.8
Mistral	2.03	3.23	1.12	0.54	0.44	0.26	0.25	0.10	3.85	4.12	114.9
Zephyr	**1.44**	2.84	**0.77**	**0.40**	**0.20**	0.29	0.16	**0.05**	**2.58**	**3.58**	98.0
GPT-3.5	0.65	1.76	0.49	0.38	0.18	0.26	0.07	0.02	1.39	2.42	84.9

Table 5.3: The average *number of errors per output* (lower is better) based on human annotators ($\mathcal{E}_{\text{hum}}$) and GPT-4 ($\mathcal{E}_{\text{gpt}}$). We also include the average number of tokens per output in the rightmost column. The results of the best open LLM are emphasized.

	Incorrect		Not Checkable		Misleading		Other		All cat.	
	$\mathcal{E}_{\text{hum}}$	$\mathcal{E}_{\text{gpt}}$	$\mathcal{E}_{\text{hum}}$	$\mathcal{E}_{\text{gpt}}$	$\mathcal{E}_{\text{hum}}$	$\mathcal{E}_{\text{gpt}}$	$\mathcal{E}_{\text{hum}}$	$\mathcal{E}_{\text{gpt}}$	$\mathcal{E}_{\text{hum}}$	$\mathcal{E}_{\text{gpt}}$
Llama 2	53.2%	80.0%	57.4%	44.8%	17.4%	**8.8%**	**7.6%**	7.6%	85.6%	94.0%
Mistral	53.6%	80.2%	49.6%	31.8%	20.6%	17.0%	13.6%	8.4%	81.2%	93.0%
Zephyr	**46.8%**	**78.0%**	**42.2%**	**25.0%**	**16.2%**	20.6%	11.6%	**4.2%**	**75.6%**	**89.4%**
GPT-3.5	38.0%	65.0%	28.8%	19.6%	12.6%	16.2%	6.2%	2.2%	60.6%	75.8%

Table 5.4: The percentage of *outputs containing at least one error* (lower is better) based on human annotators ($\mathcal{E}_{\text{hum}}$) and GPT-4 ($\mathcal{E}_{\text{gpt}}$). The results of the best open LLM are emphasized.

5.2.3 Evaluation

For evaluation, we focus on identifying *semantic errors* in model outputs. We annotate the errors on the token level, considering all the tokens in the output text as potential sources of errors.

We use two complementary referenceless evaluation methods:

- $\mathcal{E}_{\text{hum}}$: **human evaluation** based on crowdsourcing,
- $\mathcal{E}_{\text{gpt}}$: **an automatic metric** based on GPT-4.

We use similar error taxonomy and notation are we did in Section 3.2, inspired by Thomson and Reiter (2020). We use four error categories: INCORRECT$^{\text{I}}$, NOT_CHECKABLE$^{\text{NC}}$, MISLEADING$^{\text{S}}$, and OTHER$^{\text{O}}$. A summary of the token-level annotations is given in Tables 5.3 and 5.4.

Depending on the model, between 76-86% of examples contain an error according to $\mathcal{E}_{\text{hum}}$, suggesting that open LLMs make semantic errors very often. According to $\mathcal{E}_{\text{gpt}}$, the number is as high as 89-94%. The most common error type is INCORRECT$^{\text{I}}$. As shown in Table 5.3, all the open LLMs make more than **two statements contradicting the data per output on average**. See the thesis for more details about the results.

We computed the Pearson correlation coefficient between the error counts on the level of tokens, examples, and domains as follows (note that each error category was considered separately):

- For r_{domain}, we used the average error counts per domain.[6]
- For r_{example}, we used the count of errors per example.
- For r_{token}, we used binary indicators marking whether each word is labeled as an error.

We see that the correlation on the level of words is weak ($r_{\text{token}} = 0.26$) but gets better on the example level ($r_{\text{example}} = 0.52$) and even better on the domain level ($r_{\text{domain}} = 0.93$).

We also measure inter-annotator agreement between human annotators. For that, we obtained annotations from two annotators for 100 model outputs. The results are similar: the annotators agree weakly on the token level ($r_{\text{token}} = 0.36$), stronger on the example level ($r_{\text{example}} = 0.53$), and even stronger on the domain level ($r_{\text{domain}} = 0.85$). We conclude that while the details regarding error spans and categories may vary, the annotators as well as GPT-4 generally agree on the accuracy of model outputs for a given set of examples. In the future, the agreement could be improved by measuring errors on the phrase level (Vamvas and Sennrich, 2022).

[6] See Appendix F of Kasner and Dušek (2024) for the results for individual domains.

6 Conclusions

In the thesis, we set out to explore how to use language models (LMs) to improve the robustness and fluency of data-to-text (D2T) generation systems. We presented multiple findings, including:

- Simple LM-based approaches can be used for generating fluent outputs from structured data (Sections 2.1 and 5.2),
- Preprocessing the data with templates and rule-based systems can help downstream LM components (Sections 2.2, 2.3, 3.1, and 3.2),
- Constraining LM to improving text quality helps to improve the semantic accuracy of the system outputs (Sections 2.2 and 2.3),
- LMs can be also used for automatic metrics for evaluating semantic accuracy (Sections 3.1 and 3.2),
- The best way to leverage LM pretraining is to use standardized and understandable input formats (Sections 4.1, 5.1, and 5.2).

While large language models (LLMs) have revolutionized many natural language processing (NLP) tasks, their limitations and unpredictable behavior raise questions about their suitability for D2T generation. This thesis demonstrates that, despite these challenges, LLMs can still be valuable components in D2T systems when used wisely.

As users become increasingly aware of LLM limitations, it is essential to recognize that D2T generation is not solely about generating fluent texts. It is primarily about simplifying interactions for end-users, and LLM-based systems must ensure accurate outputs that meet day-to-day usage requirements. Therefore, careful consideration is needed when integrating LLMs into D2T systems.

We suggested that rather than dismissing LLMs, we should acknowledge their potential as components within larger systems. By leveraging techniques like constrained generation and connection to external tools, LLMs can be harnessed to improve D2T generation. Furthermore, recent developments hint at the possibility of integrating LLMs with code execution, allowing for automatic performance of logical operations and content selection, which are important end goals of D2T generation.

Future research should focus on developing evaluation measures that accurately reflect system improvements, moving beyond traditional benchmarks and towards ecological validity. Extrinsically evaluating systems as a whole, rather than individual components, will provide a more comprehensive understanding of their real-world impact.

We also recommend that researchers look in the data and attempt to perform the task themselves before relying on LLMs. By acknowledging the limitations of both humans and machines, we can develop more effective and practical solutions for D2T generation.[1]

[1]The content of this section (after the findings) was generated by the `meta-llama/Meta-Llama-3-8B` model prompted with: "Here is a conclusion chapter from a PhD thesis: `<content>`. Turn it into a shortened version for an extended abstract of a PhD thesis. Make it approximately 5 paragraphs.", using the original conclusion chapter from the thesis as `<content>`. The thesis author manually post-edited the generated content and checked its factual accuracy.

Bibliography

Agarwal, O. – Ge, H. – Shakeri, S. – Al-Rfou, R. Knowledge Graph Based Synthetic Corpus Generation for Knowledge-Enhanced Language Model Pre-training. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2021*, p. 3554–3565, Online, 2021. doi: 10.18653/V1/2021.NAACL-MAIN.278. Available at: https://doi.org/10.18653/v1/2021.naacl-main.278.

Attardi, G. WikiExtractor. https://github.com/attardi/wikiextractor, 2015.

Barzilay, R. – McKeown, K. R. Sentence Fusion for Multidocument News Summarization. *Comput. Linguistics*. 2005, 31, 3, p. 297–328. doi: 10.1162/089120105774321091. Available at: https://doi.org/10.1162/089120105774321091.

Bird, S. – Klein, E. – Loper, E. *Natural Language Processing With Python: Analyzing Text With the Natural Language Toolkit*. O'Reilly Media, Inc., 2009.

Botha, J. A. – Faruqui, M. – Alex, J. – Baldridge, J. – Das, D. Learning to Split and Rephrase From Wikipedia Edit History. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, p. 732–737, Brussels, Belgium, 2018. doi: 10.18653/v1/D18-1080. Available at: https://www.aclweb.org/anthology/D18-1080.

Calizzano, R. – Ostendorff, M. – Rehm, G. Ordering Sentences and Paragraphs with Pre-Trained Encoder-Decoder Transformers and Pointer Ensembles. In *DocEng '21: ACM Symposium on Document Engineering 2021*, p. 10:1–10:9, Limerick, Ireland, 2021. doi: 10.1145/3469096.3469874. Available at: https://doi.org/10.1145/3469096.3469874.

Cheng, Z. – Dong, H. – Wang, Z. – Jia, R. – Guo, J. – Gao, Y. – Han, S. – Lou, J. – Zhang, D. HiTab: A Hierarchical Table Dataset for Question Answering and Natural Language Generation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2022*, p. 1094–1110, Dublin, Ireland, 2022. doi: 10.18653/V1/2022.ACL-LONG.78. Available at: https://doi.org/10.18653/v1/2022.acl-long.78.

Devlin, J. – Chang, M. – Lee, K. – Toutanova, K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, Volume 1 (Long and Short Papers)*, p. 4171–4186, USA, 2019. doi: 10.18653/V1/N19-1423. Available at: https://doi.org/10.18653/v1/n19-1423.

Dušek, O. – Kasner, Z. Evaluating Semantic Accuracy of Data-to-Text Generation with Natural Language Inference. In *Proceedings of the 13th International Conference on Natural Language Generation, INLG 2020*, p. 131–137, Dublin, Ireland, 2020. doi: 10.18653/V1/2020.INLG-1.19. Available at: https://doi.org/10.18653/v1/2020.inlg-1.19.

Dušek, O. – Howcroft, D. M. – Rieser, V. Semantic Noise Matters for Neural Natural Language Generation. In *Proceedings of the 12th International Conference on Natural Language Generation, INLG 2019*, p. 421–426, Tokyo, Japan, 2019. doi: 10.18653/V1/W19-8652. Available at: https://aclanthology.org/W19-8652/.

Dušek, O. – Novikova, J. – Rieser, V. Evaluating the State-of-the-Art of End-to-End Natural Language Generation: The E2E NLG challenge. *Comput. Speech Lang.* 2020, 59, p. 123–156. doi: 10.1016/J.CSL.2019.06.009. Available at: https://doi.org/10.1016/j.csl.2019.06.009.

Ferreira, T. C. – Gardent, C. – Ilinykh, N. – Van Der Lee, C. – Mille, S. – Moussallem, D. – Shimorina, A. The 2020 Bilingual, Bi-Directional WebNLG+ Shared Task Overview and Evaluation Results (WebNLG+ 2020). In *Proceedings of the 3rd International Workshop on Natural Language Generation from the Semantic Web (WebNLG+)*, 2020. Available at: https://aclanthology.org/2020.webnlg-1.7/.

Gardent, C. – Shimorina, A. – Narayan, S. – Perez-Beltrachini, L. The WebNLG Challenge: Generating Text from RDF Data. In *Proceedings of the 10th International Conference on Natural Language Generation, INLG 2017, Santiago de Compostela*, p. 124–133, Spain, 2017. doi: 10.18653/V1/W17-3518. Available at: https://doi.org/10.18653/v1/w17-3518.

Gardner, M. – Grus, J. – Neumann, M. – Tafjord, O. – Dasigi, P. – Liu, N. F. – Peters, M. E. – Schmitz, M. – Zettlemoyer, L. AllenNLP: A Deep Semantic Natural Language Processing Platform. *CoRR*. 2018, abs/1803.07640. Available at: http://arxiv.org/abs/1803.07640.

Gehrmann, S. et al. The GEM Benchmark: Natural Language Generation, its Evaluation and Metrics. *CoRR*. 2021, abs/2102.01672. Available at: https://arxiv.org/abs/2102.01672.

Geva, M. – Malmi, E. – Szpektor, I. – Berant, J. DiscoFuse: A Large-Scale Dataset for Discourse-Based Sentence Fusion. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, p. 3443–3455, Minneapolis, Minnesota, jun 2019. doi: 10.18653/v1/N19-1348. Available at: https://www.aclweb.org/anthology/N19-1348.

Harkous, H. – Groves, I. – Saffari, A. Have Your Text and Use It Too! End-to-End Neural Data-to-Text Generation with Semantic Fidelity. In *Proceedings of the 28th International Conference on Computational Linguistics, COLING 2020*, p. 2410–2424, Barcelona, Spain (Online, 2020. doi: 10.18653/V1/2020.COLING-MAIN.218. Available at: https://doi.org/10.18653/v1/2020.coling-main.218.

Jiang, A. Q. et al. Mistral 7B. *CoRR*. 2023, abs/2310.06825. doi: 10.48550/ARXIV.2310.06825. Available at: https://doi.org/10.48550/arXiv.2310.06825.

Kale, M. – Rastogi, A. Text-to-Text Pre-Training for Data-to-Text Tasks. In *Proceedings of the 13th International Conference on Natural Language Generation, INLG 2020*, p. 97–102, Dublin, Ireland, 2020. doi: 10.18653/V1/2020.INLG-1.14. Available at: https://doi.org/10.18653/v1/2020.inlg-1.14.

Kasner, Z. – Dušek, O. Neural Pipeline for Zero-Shot Data-to-Text Generation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2022*, p. 3914–3932, Dublin, Ireland, 2022. doi: 10.18653/V1/2022.ACL-LONG.271. Available at: https://doi.org/10.18653/v1/2022.acl-long.271.

Kasner, Z. – Dušek, O. Data-to-Text Generation with Iterative Text Editing. In *Proceedings of the 13th International Conference on Natural Language Generation, INLG 2020*, p. 60–67, Dublin, Ireland, 2020a. doi: 10.18653/V1/2020.INLG-1.9. Available at: https://doi.org/10.18653/v1/2020.inlg-1.9.

Kasner, Z. – Dušek, O. Beyond Traditional Benchmarks: Analyzing Behaviors of Open LLMs on Data-to-Text Generation. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024. Available at: http://arxiv.org/abs/2401.10186. To appear.

Kasner, Z. – Dušek, O. Train Hard, Finetune Easy: Multilingual Denoising for RDF-to-Text Generation. In *Proceedings of the 3rd International Workshop on Natural Language Generation from the Semantic Web (WebNLG+)*, p. 171–176, Dublin, Ireland (Virtual), 12 2020b. Available at: https://aclanthology.org/2020.webnlg-1.20.

Kasner, Z. – Mille, S. – Dušek, O. Text-in-Context: Token-Level Error Detection for Table-to-Text Generation. In *Proceedings of the 14th International Conference on Natural Language Generation, INLG 2021*, p. 259–265, Aberdeen, Scotland, UK, 2021. doi: 10.18653/V1/2021.INLG-1.25. Available at: https://doi.org/10.18653/v1/2021.inlg-1.25.

Kasner, Z. – Garanina, E. – Plátek, O. – Dušek, O. TabGenie: A Toolkit for Table-to-Text Generation. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics: System Demonstrations, ACL 2023*, p. 444–455, Toronto, Canada, 2023a. doi: 10.18653/V1/2023.ACL-DEMO.42. Available at: https://doi.org/10.18653/v1/2023.acl-demo.42.

Kasner, Z. – Konstas, I. – Dušek, O. Mind the Labels: Describing Relations in Knowledge Graphs With Pretrained Models. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics, EACL 2023, Dubrovnik*, p. 2390–2407, Croatia, 2023b. doi: 10.18653/V1/2023.EACL-MAIN.176. Available at: https://doi.org/10.18653/v1/2023.eacl-main.176.

Ke, P. – Ji, H. – Ran, Y. – Cui, X. – Wang, L. – Song, L. – Zhu, X. – Huang, M. JointGT: Graph-Text Joint Representation Learning for Text Generation from Knowledge Graphs. In *Findings of the Association for Computational Linguistics: ACL/IJCNLP 2021*, ACL/IJCNLP 2021 / *Findings of ACL*, p. 2526–2538, Online Event, 2021. doi: 10.18653/V1/2021.FINDINGS-ACL.223. Available at: https://doi.org/10.18653/v1/2021.findings-acl.223.

Laha, A. – Jain, P. – Mishra, A. – Sankaranarayanan, K. Scalable Micro-planned Generation of Discourse from Structured Data. *Comput. Linguistics*. 2019, 45, 4, p. 737–763. doi: 10.1162/COLI_A_00363. Available at: https://doi.org/10.1162/coli_a_00363.

Lee, K. – He, L. – Zettlemoyer, L. Higher-Order Coreference Resolution with Coarse-to-Fine Inference. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT, Volume 2 (Short Papers)*, p. 687–692, New Orleans, Louisiana, USA, 2018. doi: 10.18653/V1/N18-2108. Available at: https://doi.org/10.18653/v1/n18-2108.

Lewis, M. – Liu, Y. – Goyal, N. – Ghazvininejad, M. – Mohamed, A. – Levy, O. – Stoyanov, V. – Zettlemoyer, L. BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, ACL 2020*, p. 7871–7880, Online, 2020. doi: 10.18653/V1/2020.ACL-MAIN.703. Available at: https://doi.org/10.18653/v1/2020.acl-main.703.

Lhoest, Q. et al. Datasets: A Community Library for Natural Language Processing. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, EMNLP 2021, Online and Punta Cana, Dominican Republic, 7-11 November, 2021*, p. 175–184, 2021. doi: 10.18653/v1/2021.emnlp-demo.21. Available at: https://doi.org/10.18653/v1/2021.emnlp-demo.21.

Liu, Y. – Ott, M. – Goyal, N. – Du, J. – Joshi, M. – Chen, D. – Levy, O. – Lewis, M. – Zettlemoyer, L. – Stoyanov, V. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *CoRR*. 2019, abs/1907.11692. Available at: http://arxiv.org/abs/1907.11692.

Malmi, E. – Krause, S. – Rothe, S. – Mirylenka, D. – Severyn, A. Encode, Tag, Realize: High-Precision Text Editing. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, EMNLP-IJCNLP 2019*, p. 5053–5064, Hong Kong, China, 2019. doi: 10.18653/V1/D19-1510. Available at: https://doi.org/10.18653/v1/D19-1510.

Mille, S. – Dasiopoulou, S. – Fisas, B. – Wanner, L. Teaching FORGe to Verbalize DBpedia Properties in Spanish. In *Proceedings of the 12th International Conference on Natural Language Generation, INLG 2019*, p. 473–483, Tokyo, Japan, 2019. doi: 10.18653/V1/W19-8659. Available at: https://aclanthology.org/W19-8659/.

Narayan, S. – Gardent, C. – Cohen, S. B. – Shimorina, A. Split and Rephrase. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing, EMNLP 2017*, p. 606–616, Copenhagen, Denmark, 2017. doi: 10.18653/V1/D17-1064. Available at: https://doi.org/10.18653/v1/d17-1064.

Ott, M. – Edunov, S. – Baevski, A. – Fan, A. – Gross, S. – Ng, N. – Grangier, D. – Auli, M. fairseq: A Fast, Extensible Toolkit for Sequence Modeling. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Demonstrations*, p. 48–53, 2019. doi: 10.18653/V1/N19-4009. Available at: https://doi.org/10.18653/v1/n19-4009.

Parikh, A. P. – Wang, X. – Gehrmann, S. – Faruqui, M. – Dhingra, B. – Yang, D. – Das, D. ToTTo: A Controlled Table-To-Text Generation Dataset. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, EMNLP 2020*, p. 1173–1186, Online, 2020. doi: 10.18653/V1/2020.EMNLP-MAIN.89. Available at: https://doi.org/10.18653/v1/2020.emnlp-main.89.

Radford, A. – Wu, J. – Child, R. – Luan, D. – Amodei, D. – Sutskever, I. Language Models Are Unsupervised Multitask Learners. *OpenAI Blog*. 2019, p. 24. Available at: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.

Reiter, E. – Thomson, C. Shared Task on Evaluating Accuracy. In *Proceedings of the 13th International Conference on Natural Language Generation, INLG 2020*, p. 227–231, Dublin, Ireland, 2020. doi: 10.18653/V1/2020.INLG-1.28. Available at: https://doi.org/10.18653/v1/2020.inlg-1.28.

Thomson, C. – Reiter, E. A Gold Standard Methodology for Evaluating Accuracy in Data-To-Text Systems. In *Proceedings of the 13th International Conference on Natural Language Generation, INLG 2020*, p. 158–168, Dublin, Ireland, 2020. doi: 10.18653/V1/2020.INLG-1.22. Available at: https://doi.org/10.18653/v1/2020.inlg-1.22.

TogetherAI. Preparing for the Era of 32K Context: Early Learnings and Explorations. https://www.together.ai/blog/llama-2-7b-32k, 2023. Accessed on January 2, 2024.

Touvron, H. et al. Llama 2: Open Foundation and Fine-Tuned Chat Models. *CoRR*. 2023, abs/2307.09288. doi: 10.48550/ARXIV.2307.09288. Available at: https://doi.org/10.48550/arXiv.2307.09288.

Tunstall, L. – Beeching, E. – Lambert, N. – Rajani, N. – Rasul, K. – Belkada, Y. – Huang, S. – Werra, L. – Fourrier, C. – Habib, N. – Sarrazin, N. – Sanseviero, O. – Rush, A. M. – Wolf, T. Zephyr: Direct Distillation of LM Alignment. *CoRR*. 2023, abs/2310.16944. doi: 10.48550/ARXIV.2310.16944. Available at: https://doi.org/10.48550/arXiv.2310.16944.

Vamvas, J. – Sennrich, R. As Little as Possible, as Much as Necessary: Detecting Over- and Undertranslations with Contrastive Conditioning. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers), ACL 2022*, p. 490–500, Dublin, Ireland, 2022. doi: 10.18653/V1/2022.ACL-SHORT.53. Available at: https://doi.org/10.18653/v1/2022.acl-short.53.

Wang, T. – Wan, X. Hierarchical Attention Networks for Sentence Ordering. In *The Thirty-Third AAAI Conference on Artificial Intelligence, AAAI 2019, The Thirty-First Innovative Applications of Artificial Intelligence Conference, IAAI 2019, The Ninth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2019*, p. 7184–7191, Honolulu, Hawaii, USA, 2019. doi: 10.1609/AAAI.V33I01.33017184. Available at: https://doi.org/10.1609/aaai.v33i01.33017184.

Williams, A. – Nangia, N. – Bowman, S. R. A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2018, Volume 1 (Long Papers)*, p. 1112–1122, New Orleans, Louisiana, USA, 2018. doi: 10.18653/V1/N18-1101. Available at: https://doi.org/10.18653/v1/n18-1101.

Wolf, T. – Debut, L. – Sanh, V. – Chaumond, J. – Delangue, C. – Moi, A. – Cistac, P. – Rault, T. – Louf, R. – Funtowicz, M. – Brew, J. HuggingFace's Transformers: State-of-the-Art Natural Language Processing. *CoRR*. 2019, abs/1910.03771. Available at: http://arxiv.org/abs/1910.03771.

List of Publications

Kasner, Z. – Dušek, O. Train Hard, Finetune Easy: Multilingual Denoising for RDF-to-Text Generation. In *Proceedings of the 3rd International Workshop on Natural Language Generation from the Semantic Web (WebNLG+)*, p. 171–176, Dublin, Ireland (Virtual), 12 2020b. Available at: https://aclanthology.org/2020.webnlg-1.20

- Citations (without self-citations): 9

Kasner, Z. – Dušek, O. Data-to-Text Generation with Iterative Text Editing. In *Proceedings of the 13th International Conference on Natural Language Generation, INLG 2020*, p. 60–67, Dublin, Ireland, 2020a. doi: 10.18653/V1/2020.INLG-1.9. Available at: https://doi.org/10.18653/v1/2020.inlg-1.9

- Citations (without self-citations): 17

Kasner, Z. – Dušek, O. Neural Pipeline for Zero-Shot Data-to-Text Generation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2022*, p. 3914–3932, Dublin, Ireland, 2022. doi: 10.18653/V1/2022.ACL-LONG.271. Available at: https://doi.org/10.18653/v1/2022.acl-long.271

- Citations (without self-citations): 21

Dušek, O. – Kasner, Z. Evaluating Semantic Accuracy of Data-to-Text Generation with Natural Language Inference. In *Proceedings of the 13th International Conference on Natural Language Generation, INLG 2020*, p. 131–137, Dublin, Ireland, 2020. doi: 10.18653/V1/2020.INLG-1.19. Available at: https://doi.org/10.18653/v1/2020.inlg-1.19

- Citations (without self-citations): 47

Kasner, Z. – Mille, S. – Dušek, O. Text-in-Context: Token-Level Error Detection for Table-to-Text Generation. In *Proceedings of the 14th International Conference on Natural Language Generation, INLG 2021*, p. 259–265, Aberdeen, Scotland, UK, 2021. doi: 10.18653/V1/2021.INLG-1.25. Available at: https://doi.org/10.18653/v1/2021.inlg-1.25

- Citations (without self-citations): 6

Kasner, Z. – Garanina, E. – Plátek, O. – Dušek, O. TabGenie: A Toolkit for Table-to-Text Generation. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics: System Demonstrations, ACL 2023*, p. 444–455, Toronto, Canada, 2023a. doi: 10.18653/V1/2023.ACL-DEMO.42. Available at: https://doi.org/10.18653/v1/2023.acl-demo.42

- Citations (without self-citations): 2

Kasner, Z. – Konstas, I. – Dušek, O. Mind the Labels: Describing Relations in Knowledge Graphs With Pretrained Models. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics, EACL 2023, Dubrovnik*, p. 2390–2407, Croatia, 2023b. doi: 10.18653/V1/2023.EACL-MAIN.176. Available at: https://doi.org/10.18653/v1/2023.eacl-main.176

- The analysis of verbalizing relations in knowledge graphs with pretrained language models (Section 5.1).
- Citations (without self-citations): 3

Kasner, Z. – Dušek, O. Beyond Traditional Benchmarks: Analyzing Behaviors of Open LLMs on Data-to-Text Generation. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024. Available at: http://arxiv.org/abs/2401.10186. To appear

- The analysis of data-to-text generation with open large language models (Section 5.2).
- Citations (without self-citations): 2

Only publications relevant to this thesis are included. The number of citations was computed using Semantic Scholar API. Total number of citations of publications related to the topic of the thesis (without self-citations) by June 14, 2024: **107**.